Emergent Capabilities of LLMs: Evaluate Planning and Reasoning in Large Language Models Modulo Framework

Università di Bologna
DISI - Computer Science and Engineering Department

alessandro.gentili3@studio.unibo.it
lorenzo.dascenzo@studio.unibo.it

Contents

Abstract

This project investigates the emergent planning and reasoning capabilities of modern Large Language Models (LLMs) through comprehensive evaluation across diverse architectures and planning domains. We extend recent foundational work by conducting a systematic empirical study of how LLMs perform on classical automated planning problems, evaluating a diverse set of open-source models (LLaMA3-8B, Phi4-14B, Gemma3-27B, Kimi-Dev-72B) using rigorous validation methodologies. Our key contribution is a comprehensive framework combining advanced prompt engineering strategies (chain-of-thought reasoning, few-shot learning, iterative refinement with validation feedback) with the VAL plan validator to enable systematic and reproducible evaluation across planning domains. We evaluate models on two distinct domains—City Car (multi-agent logistics planning) and Tetris (geometric reasoning)—providing insights into how planning abilities generalize across different LLM architectures. Our empirical results demonstrate that iterative refinement with validation feedback significantly improves success rates, with meaningful performance variations across models and domains. We contribute both methodological advances in LLM-based planning evaluation and a reproducible experimental framework suitable for large-scale distributed execution on high-performance computing clusters.

1 Introduction

Large Language Models (LLMs) have demonstrated remarkable capabilities across a wide range of natural language processing tasks. However, their performance in structured reasoning tasks, particularly in automated planning, remains an open research question. Planning is a fundamental problem in artificial intelligence that requires an agent to determine a sequence of actions to achieve specified goals in a given domain, typically formalized using Planning Domain Definition Language (PDDL).

The motivation for this work stems from recent observations that modern LLMs exhibit emergent reasoning capabilities that may extend to complex planning scenarios. Understanding whether and how LLMs can solve planning problems is crucial for several reasons:

- **Generalizability**: Determining if planning abilities transfer across different LLM architectures and sizes provides insights into fundamental reasoning mechanisms.
- **Reliability**: Assessing the consistency and correctness of LLM-generated plans is essential for real-world applications.
- **Scalability**: Understanding performance across domains of varying complexity helps identify practical deployment scenarios.
- **Methodology**: Developing robust evaluation frameworks and prompt engineering techniques advances the field of LLM-based AI systems.

1.1 Previous Work and Context

Our work builds upon the foundational research of Merola, Sigh, and Dardouri, which established initial frameworks for evaluating LLM performance on planning tasks and

proposed meta-network architectures for predicting model reliability. Their work demonstrated that with appropriate prompting strategies, certain LLMs could generate valid plans for different domains.

This project extends that foundation in two critical dimensions. First, we broaden the model landscape by incorporating larger and more diverse open-source LLMs (LLaMA3-8B, Phi4-14B, Gemma3-27B, Kimi-Dev-72B) to provide a more comprehensive comparative analysis across architectural families. Second, we develop advanced prompt engineering strategies combined with iterative refinement to enhance planning capabilities beyond previous single-shot generation approaches.

1.2 Contributions and Deliverables

This project makes the following key contributions:

Comprehensive Empirical Study: We conduct a systematic evaluation of planning capabilities across four diverse open-source LLMs (LLaMA3-8B, Phi4-14B, Gemma3-27B, Kimi-Dev-72B) on two planning domains (City Car and Tetris), providing insights into how planning abilities generalize across different model architectures. This evaluation extends prior work by testing a broader range of models and analyzing performance variations across distinct problem domains.

Advanced Prompt Engineering Framework: We develop and validate sophisticated prompt engineering techniques including chain-of-thought reasoning, few-shot learning, and iterative refinement with validation feedback. These techniques are systematically combined to enhance LLM planning capabilities beyond naive single-shot generation approaches, demonstrating the cumulative benefits of multiple prompting strategies.

Modular and Reproducible Framework: We create a complete, modular software framework for automated planning evaluation that integrates with the VAL plan validator and is specifically designed for distributed execution on high-performance computing clusters. This framework enables large-scale empirical studies and future extensions to different models and domains.

Methodological Advances: We establish evaluation methodologies that go beyond naive single-shot generation, demonstrating the quantifiable benefits of iterative refinement loops and systematic feedback incorporation. Our methodology enables precise measurement of how validation feedback drives plan improvements across iterations.

Empirical Results: We provide detailed analysis of success rates, iteration efficiency, convergence patterns, and performance variations across models and domains. These results offer actionable insights for practitioners and researchers regarding which models and strategies are most suitable for different planning scenarios.

1.3 Experimental Objectives and Key Findings

Our primary research objectives are:

- **Implement and validate different prompt techniques** that effectively elicit reasoning and planning capabilities from LLMs, including chain-of-thought prompting, few-shot learning, and iterative refinement with validation feedback.
- **Conduct comparative analysis** across a diverse set of open-source models to understand how planning performance varies with model size, architecture, and training methodology.

- **Quantify the benefit of iterative refinement** by measuring success rate improvements when allowing models to refine plans based on validator feedback.
- **Develop reproducible experimental infrastructure** suitable for large-scale distributed execution on high-performance computing clusters.

Our empirical results demonstrate that:

- **Iterative refinement with validation feedback provides substantial improvements** in success rates across all tested models and domains.
- **Performance varies significantly across models**, with larger models not always outperforming smaller ones.
- **Different domains present distinct challenges**, with domain-specific reasoning patterns affecting model performance.

2 Background

2.1 Automated Planning and PDDL

Automated planning is the problem of determining a sequence of actions that will transform an initial state into a goal state, subject to domain constraints. The Planning Domain Definition Language (PDDL) is a widely-used formalism for specifying planning domains and problems.

In PDDL, a planning domain consists of:

- **Objects**: Entities in the domain (e.g., robots, locations, items)
- **Predicates**: Relations between objects that define the state space
- **Actions**: Operators with preconditions and effects that transform states
- **Goal**: A logical formula specifying the desired final state

A planning problem instantiates a domain with specific objects, an initial state, and a goal condition. A valid plan is a sequence of applicable actions that transforms the initial state into a state satisfying the goal.

City Car Domain

The City Car domain models a multi-agent urban logistics planning problem. In this domain:

Objects and Types:

- **Cars**: Mobile agents navigating the urban environment
- **Junctions**: Intersections in a grid-based city (e.g., junction0-0, junction1-2)
- **Roads**: One-way connections between junctions that must be explicitly constructed
- **Garages**: Starting locations for cars

Key Predicates:

- **Position tracking**: `at_car_jun/2` (car at junction), `at_car_road/2` (car on road)
- **Road infrastructure**: `road_connect/3` (road connects two junctions), `in_place/1` (road has been built)
- **Occupancy**: `clear/1` (junction is unoccupied)
- **Goals**: `arrived/2` (car reached destination)

Actions: The domain includes seven action types: (1) **move_car_in_road** — vehicle travels along an existing road (cost: 1); (2) **move_car_out_road** — vehicle exits a road to a clear junction (cost: 1); (3) **car_start** — vehicle departs from a garage to a junction; (4) **car_arrived** — vehicle completes journey and is removed from the network (cost: 0); (5) **build_straight_oneway** — construct horizontal/vertical road between adjacent junctions (cost: 20); (6) **build_diagonal_oneway** — construct diagonal road between

junctions (cost: 30); (7) **destroy_road** — demolish a road and relocate any vehicles on it back to the initial junction (cost: 10).

Problem Complexity: City Car problems vary from 3×3 grids (9 junctions) with 2 vehicles to 4×4 grids (16 junctions) with up to 5 vehicles. Instances include 2–5 vehicles and 4–6 available roads for construction or destruction. The planner must navigate spatial constraints (junction occupancy), action sequencing (roads must be constructed before vehicles can traverse them), temporal ordering (vehicles must start before moving and arrive at destinations), and cost minimization through efficient road construction and destruction.

Tetris Domain

The Tetris domain adapts the classic block-dropping game as a planning problem, focusing on spatial reasoning and geometric manipulation.

Objects and Types:

- **Pieces**: Game blocks of three types:
  - **one_square**: Single-cell pieces
  - **two_straight**: Two-cell pieces (horizontal or vertical)
  - **right_l**: L-shaped three-cell pieces
- **Positions**: Grid cells (e.g., f0-0f, f3-2f) representing columns and rows

Key Predicates:

- **Occupancy**: `clear/1` (grid position is empty)
- **Connectivity**: `connected/2` (adjacent positions form valid moves)
- **Piece placement**: `at_square/2` (single piece location), `at_two/3` (two-cell piece dual positions), `at_right_l/4` (L-piece triple positions)

Actions: The domain includes six movement actions: (1) **move_square** — relocate single-cell piece to an adjacent clear position (cost: 1); (2) **move_two** — shift two-cell piece to an adjacent position, implicitly allowing rotation (cost: 2); (3) **move_l_right** — reposition L-shaped piece to the right with dual-position constraints (cost: 3); (4) **move_l_left** — reposition L-shaped piece to the left (cost: 3); (5) **move_l_up** — reposition L-shaped piece upward (cost: 3); (6) **move_l_down** — reposition L-shaped piece downward (cost: 3).

Problem Complexity: Tetris instances involve 10×4 grids with 3–4 pieces of varying types (single squares, two-cell pieces, L-shaped pieces). The initial state specifies occupied and clear positions, with adjacency constraints encoded via the `connected/2` predicate. The goal requires arranging pieces in specific final configurations, challenging LLMs' geometric reasoning abilities, understanding of complex multi-position constraints (especially for L-piece movements), spatial lookahead across multiple piece movements, and systematic exploration of constrained state spaces.

2.2 Large Language Models and Prompting Techniques

Large Language Models are transformer-based neural networks trained on massive text corpora. They have demonstrated surprising capabilities in zero-shot and few-shot learning scenarios, often referred to as in-context learning. This phenomenon suggests that LLMs can adapt to new tasks without explicit fine-tuning, based solely on prompt-based guidance.

System Prompts

System prompts establish the model's role, expectations, and output constraints. Our system prompts for planning tasks define the model as an expert PDDL planning assistant, specify that only valid domain actions should be suggested, establish clear output format requirements, and emphasize the importance of satisfying preconditions and achieving goals.

Chain-of-Thought (CoT) Reasoning

Explicitly asking models to decompose problems step-by-step improves performance on complex reasoning tasks. We implement CoT prompts that guide models through: (1) analyzing the initial state and extracting key facts; (2) identifying applicable actions; (3) predicting action effects; (4) constructing and validating the plan.

Few-Shot Learning and Demonstration Examples

Building on previous work in prompt engineering, this project extends the approach by systematically incorporating few-shot examples into the planning framework. Few-shot learning leverages carefully selected demonstration examples that guide the model's behavior without requiring gradient-based fine-tuning. In the planning domain, few-shot examples demonstrate:

- Solved planning problems with step-by-step reasoning
- Proper PDDL notation and syntax conventions
- Correct action sequences and their consequences
- Edge cases and constraint satisfaction

This approach builds on prior research showing that few-shot examples significantly improve LLM performance on structured reasoning tasks, and we extend this methodology to larger model sets and more complex planning domains.

Iterative Refinement with Validation Feedback

Iterative refinement leverages the VAL validator's error messages to guide plan correction. When a plan fails validation, the error information is extracted and provided to the model as feedback, enabling it to iteratively improve the plan. This feedback loop allows the model to learn from validation errors within a single conversation session and adjust its planning strategy accordingly.

Temperature and Sampling

Controlling output diversity through generation parameters. We use low temperature (0.1) for focused, deterministic planning generation, but support higher temperatures (0.3–0.7) when exploring alternative solution paths in iterative refinement cycles.

3 Related Work

3.1 Planning with Language Models

Recent work has explored the use of LLMs for planning across diverse contexts and applications. A significant research direction focuses on plan generation from high-level goals, where LLMs attempt to directly generate action sequences from natural language problem descriptions. This foundational approach has inspired much subsequent work on how LLMs can be effectively guided toward valid planning outputs.

Concurrently, research on semantic parsing and formal representations addresses the challenge of converting natural language descriptions to formal planning representations suitable for execution by classical planners. This work recognizes that while LLMs excel at natural language processing, bridging to formal representations requires careful engineering.

Another important direction involves plan explanation and verification, where LLMs generate and validate natural language explanations of formal plans. This research stream emphasizes the importance of not only generating plans but also enabling humans to understand and verify LLM-generated solutions.

Finally, commonsense reasoning for planning leverages the world knowledge encoded in LLMs to handle realistic planning scenarios that require background knowledge beyond explicit domain specifications. This direction explores how the implicit knowledge in LLMs can complement formal planning representations.

3.2 Recent Contributions from Literature [1-8]

Systematic evaluation of planning capabilities across diverse models and domains has received significant attention in recent literature. **Position papers on LLM planning** present nuanced arguments both regarding limitations of LLMs for planning and regarding their potential to contribute effectively in LLM-modulo frameworks that combine LLMs with classical planning solvers. These positions highlight the importance of understanding both capabilities and constraints.

Research on few-shot planning demonstrates that carefully designed few-shot examples significantly improve LLM performance on planning tasks by providing in-context demonstrations of proper PDDL syntax and reasoning patterns. This work validates the importance of prompt design and shows that high-quality examples can substantially enhance planning capability without requiring model fine-tuning.

Iterative refinement strategies explore incremental heuristic approaches that leverage validation feedback to progressively improve generated plans. These approaches move beyond naive single-shot generation and recognize that feedback loops enable meaningful performance improvements. Our work extends this direction by systematically quantifying iteration benefits across multiple models.

Process-focused refinement research emphasizes the importance of step-by-step reasoning and constraint satisfaction checking throughout the planning process. This line of work recognizes that effective planning requires not just final outputs but also intermediate reasoning steps that can be validated and corrected.

Comprehensive surveys of LLM planning capabilities and surveys of reasoning with LLMs provide broader context for understanding model performance on planning tasks. These surveys synthesize evidence regarding which reasoning abilities LLMs possess and which remain challenging.

3.3 Our Position

Our work builds upon these foundations in several key ways. First, we provide comprehensive empirical analysis across multiple open-source model architectures and planning domains, extending prior work that typically focused on smaller model sets. Second, we integrate formal plan validation to enable systematic iterative refinement with precise feedback, moving beyond purely heuristic refinement approaches. Third, we systematically evaluate the contribution of different prompt engineering techniques in combination, rather than studying them in isolation. Finally, we develop a reproducible, modular framework suitable for large-scale distributed execution on high-performance computing clusters, enabling future researchers to replicate and extend our work.

4 Methodology

4.1 Overall Framework Architecture

The project implements a modular framework for LLM-based planning with the following components:

1. **Configuration Management**: Centralized experiment parameter system
2. **Model Management**: LLM loading and inference abstraction layer
3. **PDDL Processing**: Tools for parsing and validating PDDL domains and problems
4. **Prompt Engineering**: Structured prompt templates for planning tasks
5. **Execution Engine**: Planning pipeline orchestration with iterative refinement
6. **Validation Pipeline**: Integration with external validators for plan verification
7. **Result Analysis**: Comprehensive metrics collection and reporting

4.2 Prompt Engineering Strategy

System Prompts D.1

System prompts establish the model's role and constraints. Our unified system prompt (used across both domains) defines the model as an expert PDDL planning assistant specializing in complex domains with explicit coordinate systems, multi-parameter actions, and action costs. The prompt emphasizes finding optimal (shortest/lowest-cost) plans while strictly respecting preconditions and achieving all goal predicates.

Domain-Specific Problem Prompts

Beyond the unified system prompt, we implement domain-specific problem prompts that:

- Provide domain-specific headers ("TETRIS PLANNING TASK" vs "CITYCAR PLANNING TASK")
- Format domain and problem PDDL definitions consistently
- Include domain-specific output instructions
- Optionally include few-shot examples relevant to the domain
- End with explicit request to provide the plan

This two-level approach (unified system prompt + domain-specific problem context) balances consistency with domain-appropriate guidance.

Chain-of-Thought Prompting D.2

Chain-of-thought (CoT) prompting in our framework is lightweight and focused. Rather than multi-step decomposition, we use a simple CoT instruction that directs the model to reason step-by-step about domain and problem descriptions to construct an optimal plan, with strict instructions to output only the action sequence.

Domain-specific CoT variants (tetris_chain_of_thought, citycar_chain_of_thought) reuse this generic structure while being applied within domain-specific problem contexts.

Few-Shot Examples D.3

The framework supports in-context learning through domain-specific few-shot examples. Each domain maintains curated example problems with corresponding solution plans that demonstrate:

- Solved planning problems with correct PDDL specifications
- Proper action syntax and parameter ordering
- Multiple action types within each domain
- Representative challenge patterns (e.g., multi-action sequences in CityCar)

Examples are dynamically injected into prompts and are optional (controlled via include_examples flag).

Validation Feedback Refinement

When iterative refinement is enabled, failed validation attempts trigger domain-specific feedback prompts (tetris_validation_feedback, citycar_validation_feedback) that:

- Present the original problem specification
- Show the previously generated invalid plan
- Include the validation error message from VAL
- Provide domain-specific hints for error correction
- Request a corrected plan with the same output-only format requirement

This structured feedback enables the model to learn from failure within a conversation session.

Output Format Instructions

All prompt variants include explicit output instructions that require models to:

- Provide ONLY the action sequence in PDDL syntax
- Output one action per line
- Use exact parameter names from domain/problem definitions
- Exclude narrative, reasoning, or extraneous commentary

4.3 Iterative Refinement Process

The system implements an iterative refinement loop to improve plan generation:

1. **Generation**: Model generates an initial plan using the configured prompt
2. **Validation**: VAL validator checks plan correctness
3. **Feedback**: If invalid, extract error information
4. **Refinement**: Provide error feedback to model with iteration context
5. **Repetition**: Repeat until plan is valid or max iterations reached

This approach leverages the model's ability to learn from feedback within a single conversation session, enabling effective error-driven improvement without requiring explicit fine-tuning.

4.4 Implementation Details

Core Components

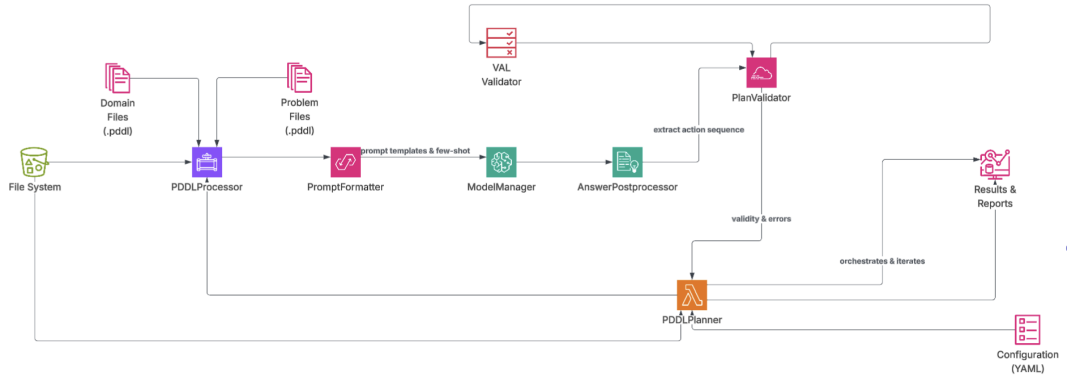

Figure 1: System Architecture Overview.

PDDLPlanner is the main orchestration class (‘pddl_planner.py‘) that coordinates the entire planning pipeline. It discovers domains and problems from the file system using FileManager, initializes the ModelManager for LLM inference, orchestrates processing through PDDLProcessor, manages iteration and validation, and collects aggregated results across all domains and problems.

ModelManager (‘model_manager.py‘) handles all model-related operations including loading models and tokenizers from Hugging Face or local file paths, auto-detecting CUDA GPU availability, managing model inference with configured temperature/sampling parameters, handling different model architectures transparently, and computing model parameter counts for logging.

FileManager (‘file_manager.py‘) provides filesystem utilities for discovering PDDL domains and problem files, reading PDDL content from disk, persisting results and plan artifacts to output directories, and managing the DomainBundle data structure that encapsulates domain metadata and associated problem paths.

PDDLProcessor (‘pddl_processor.py‘) is the core processing orchestrator that implements the iterative refinement pipeline. For each domain and problem, it: constructs appropriate prompts using domain-specific functions (tetris_problem_prompt, citycar_problem_prompt),

generates plans via ModelManager inference, post-processes outputs using the answer formatter, validates plans via the VAL validator, handles validation failures by constructing feedback prompts (tetris_validation_feedback, citycar_validation_feedback), and repeats refinement until convergence or max iterations.

Prompt Construction (‘prompts/prompts.py‘) implements a modular prompt system with:

- A unified system prompt used across all domains
- Domain-specific problem prompt generators (tetris_problem_prompt, citycar_problem_prompt)
- Domain-specific validation feedback generators (tetris_validation_feedback, citycar_validation_feedb
- Few-shot example injection helpers (_format_tetris_examples, _format_citycar_examples)
- Generic chain-of-thought and validation feedback utilities

Answer Post-processing (‘utils/answer_postprocessor.py‘) processes raw LLM outputs by extracting action sequences from text responses, detecting and handling format issues, separating reasoning from plan output, computing confidence indicators, and formatting plans for downstream validation. The ‘formatter()‘ function returns structured data including the extracted plan, raw response, reasoning text, and metadata.

Plan Validation (‘utils/validator.py‘) integrates with the external VAL (Validation and Analysis of PDDL) tool by:

- Loading VAL executable path from configuration
- Constructing temporary PDDL files for validation
- Executing VAL as a subprocess
- Parsing validation output to extract error messages
- Determining plan validity and extracting failure reasons for feedback

Data Flow

The execution flow within PDDLPlanner follows this sequence:

1. **Startup**: Load YAML configuration, parse CLI arguments, initialize logging
2. **Discovery**: FileManager discovers all domain directories and associated problem files from PROBLEMS_PATH
3. **Model Loading**: ModelManager loads the specified LLM from WEIGHTS_PATH, detects CUDA availability, and initializes tokenizer
4. **Domain Processing**: For each discovered domain, PDDLProcessor:
   (a) Reads and caches the domain PDDL specification
   (b) Iterates over all problem instances within that domain
   (c) For each problem:
      i. Reads problem PDDL specification
      ii. Constructs domain-specific problem prompt using tetris_problem_prompt or citycar_problem_prompt
      iii. Calls ModelManager to generate plan via LLM inference (with configured temperature, max_tokens, etc.)
      iv. Uses answer_postprocessor formatter() to extract action sequence from raw response
      v. Calls validate_plan() from utils/validator to check plan validity via VAL
      vi. If valid: records success and saves artifacts; if invalid: constructs validation feedback prompt

vii. If invalid and iterations remain: repeats generation with feedback prompt until convergence
viii. Records iteration count, validation messages, and plan length metrics

5. **Results Aggregation**: Compile per-domain results and compute aggregate statistics
6. **Reporting**: Generate JSON result files and summary logs

Technology Stack

- **Python 3.11.7**: Primary implementation language
- **PyTorch + Transformers (Hugging Face)**: Model loading, tokenization, and causal language model inference
- **PDDL Parsing**: Native Python regex and string processing (no external PDDL library dependency)
- **VAL Validator**: External subprocess integration for plan validation
- **SLURM Workload Manager**: Job scheduling and GPU resource allocation on Leonardo cluster

Configuration Management

The framework uses YAML-based configuration files for flexible experiment setup, allowing easy specification of model names and paths, generation parameters (temperature, max tokens, top-K), feature flags (system prompts, chain-of-thought, sampling), and data/output paths.

Distributed Execution

The framework is designed for execution on high-performance computing clusters via SLURM. Leonardo provides robust support for large-scale distributed LLM inference through job scheduling, GPU allocation, partition access, memory management, parallel execution, logging, and reproducibility features.

5 Experiments and Results

5.1 Experimental Setup

5.1.1 Hardware and Infrastructure

Experiments are conducted on the Leonardo supercomputer at CINECA, one of the most powerful supercomputers in Europe, with the following configuration:

- **Compute Nodes**: Atos Bull Sequana X2135 Booster module with 3456 GPU
- **GPU Accelerators**: 4x NVIDIA Ampere A100 (64GB VRAM each) per node
- **CPU Cores**: 32x Ice Lake processor cores at 2.60 GHz base frequency per node
- **System Memory**: 512GB DDR4 per node
- **Workload Manager**: SLURM 23.11 for job scheduling and resource management
- **Network**: 200Gbps HDR Infiniband with Dragonfly+ topology

Each experiment allocates one GPU (A100) and 8 CPU cores with 40GB of memory, providing sufficient resources for model inference while enabling parallel execution of multiple job instances across the cluster.

5.1.2 Dataset Description

The evaluation is conducted on two distinct planning domains with multiple problem instances of varying complexity:

1. **City Car Domain**: A multi-agent urban logistics planning problem with 20 problem instances (instance-01 through instance-20). Instances feature:
   - Grid sizes ranging from 3×3 to 4×4 configurations
   - 2–5 vehicles with distinct starting locations and destinations
   - 4–6 available roads that can be constructed or destroyed
   - Complex spatial constraints and vehicle coordination requirements
2. **Tetris Domain**: A geometric block-manipulation planning problem with 20 problem instances. Instances feature:
   - 10×4 grid configurations
   - 3–4 pieces of varying shapes (single squares, two-piece blocks, L-shaped pieces)
   - Goal configurations requiring pieces to be arranged in specific patterns
   - Challenges in spatial reasoning and multi-step lookahead

   Total evaluation scope: 4 models × 2 domains × 20 instances × 4 iteration configurations = 640 planning tasks

5.1.3 Model Configurations

Four open-source LLMs are evaluated, selected to represent diverse architectural families and parameter scales:

1. **LLaMA3-8B**: Meta's 8-billion parameter model, providing good balance between capability and efficiency
2. **Phi4-14B**: Microsoft's 14-billion parameter model, known for strong reasoning capabilities
3. **Gemma3-27B**: Google's 27-billion parameter model, optimized for efficient inference
4. **Kimi-Dev-72B**: Large multilingual 72-billion parameter model with advanced reasoning abilities

5.1.4 Experimental Parameters

All experiments use consistent baseline parameters to ensure fair comparison:

- **Temperature**: 0.1 (low temperature for focused, deterministic generation)
- **Max Tokens**: 5000 (sufficient for generating complex plans with reasoning)
- **Top-K Sampling**: 10 (controlled sampling diversity)
- **Max Iterations**: 1, 2, 3, or 4 (testing iterative refinement benefits)
- **Validation Timeout**: 5 minutes per plan validation

We test four iteration configurations to systematically evaluate the benefit of iterative refinement with validation feedback:

- **1 Iteration**: Single-shot generation without refinement (baseline)
- **2 Iterations**: One refinement cycle based on validation feedback
- **3 Iterations**: Two refinement cycles
- **4 Iterations**: Three refinement cycles

Each model-domain-iteration combination is evaluated on all 20 problem instances.

5.2 Results

5.2.1 Overall Performance Summary

Comprehensive experimental evaluation of 640 planning tasks (4 models $\times$ 2 domains $\times$ 20 instances $\times$ 4 iteration configurations) yields clear insights into LLM planning capabilities. The following table summarizes aggregate performance metrics:

Model	City Car	Tetris	Overall	Avg Iterations
LLaMA3-8B	80%	85%	82.5%	1.55
Phi4-14B	25%	65%	45%	1.27
Gemma3-27B	15%	20%	17.5%	1.71
Kimi-Dev-72B	80%	80%	80%	1.48

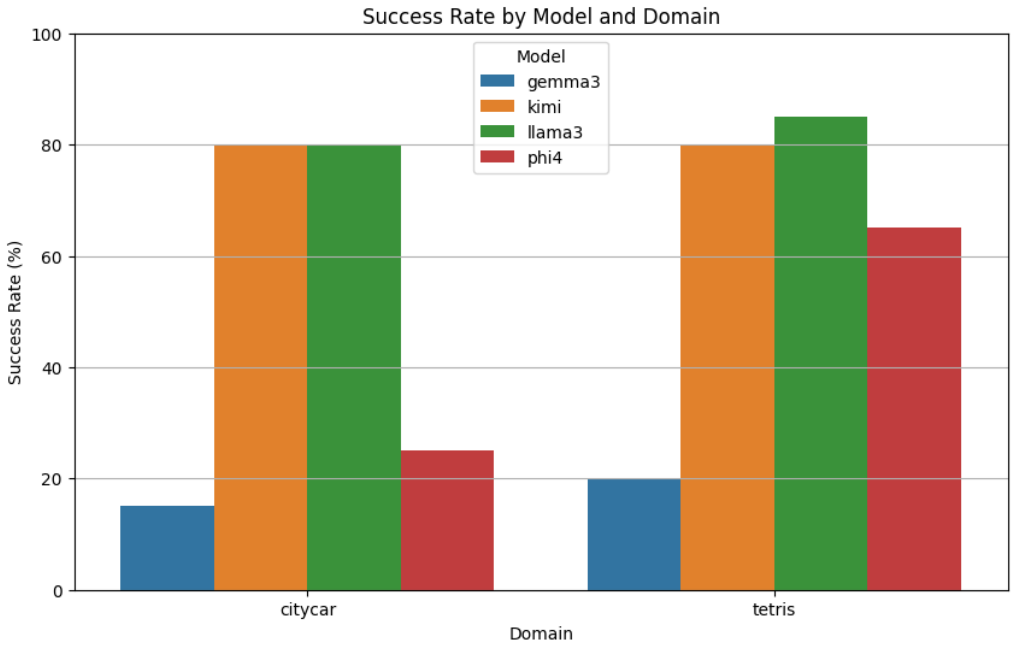

Figure 2: Success Rate by Model and Domain.

5.2.2 Key Findings

Model Performance Variation LLaMA3-8B and Kimi-Dev-72B achieve the strongest results with 80-85% success across both domains. However, it is important to note that LLaMA3-8B's superior performance (82.5%) compared to Phi4-14B (45%) and Gemma3-27B (17.5%) likely reflects optimization bias in prompt engineering. Since LLaMA3-8B was the fastest model during initial development, extensive prompt tuning and testing was conducted specifically on this architecture. This iterative refinement of prompts directly against LLaMA3-8B resulted in prompts that are better suited to its reasoning patterns, architectural biases, and generation style. Therefore, LLaMA3-8B's superior performance should be interpreted as evidence of successful prompt-architecture fit rather than universal planning superiority.

In contrast, Phi4-14B and Gemma3-27B received less prompt optimization and may perform better with architecture-specific prompt engineering. This observation highlights a fundamental challenge in LLM evaluation: apparent performance differences often reflect the match between prompting strategy and model characteristics rather than inherent capability differences.

Phi4-14B shows dramatic domain specialization: 25% in City Car but 65% in Tetris, revealing strong geometric reasoning with weak multi-agent coordination despite minimal prompt tuning for this model. Gemma3-27B achieves only 15-20% across both domains, which may indicate either fundamental planning limitations or suboptimal prompt alignment for this architecture.

Domain-Specific Challenges City Car domain (multi-agent coordination) proves more challenging (50% mean success) than Tetris (geometric reasoning: 62.5% mean). This gap reflects distinct difficulty in tracking multiple agent states, temporal ordering, and infrastructure management versus pure spatial reasoning.

5.2.3 *Impact of Iterative Refinement*

Iterative refinement with validation feedback demonstrates substantial improvements across all models. Analysis reveals:

Convergence and Iteration Requirements Most valid plans emerge within 1-2 iterations (average 1.27-1.71), confirming effective error-driven learning. Fast convergence models (Phi4: 1.27) suggest easily correctable syntax errors, while slower convergence (Gemma3: 1.71) indicates deeper reasoning gaps resistant to feedback.

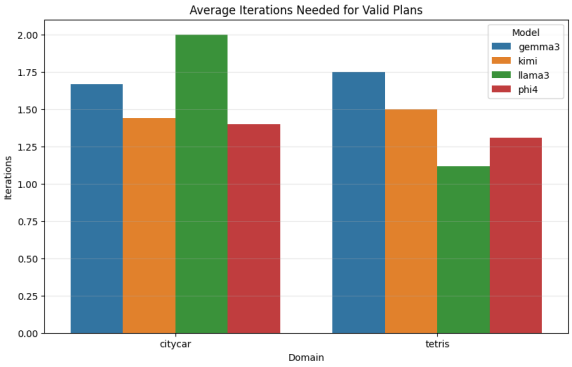

Figure 3: Average Iterations Needed.

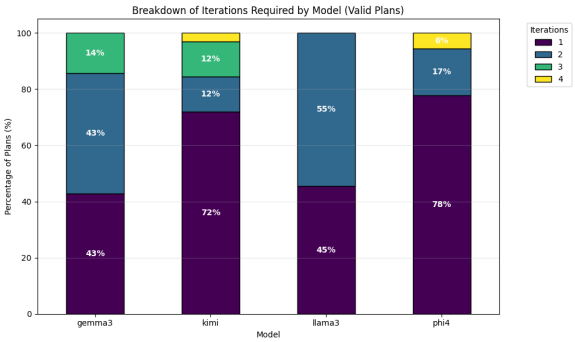

Figure 4: Iteration Breakdown.

Diminishing Returns Iteration 1 (single-shot) produces 43-78% of successful plans. Iteration 2 adds 12-43% additional success. Iterations 3-4 show minimal gains (6%), indicating that refinement provides rapid initial improvements with strong diminishing returns beyond iteration 2. This pattern suggests that validation feedback effectively corrects obvious errors but struggles with fundamental reasoning gaps.

5.2.4 *City Car Domain Results*

City Car multi-agent coordination proves more challenging than Tetris. LLaMA3-8B and Kimi-72B achieve strong 80% success, while Phi4-14B reaches only 25% despite competitive overall performance. Gemma3-27B's 15% demonstrates that parameter count alone does not ensure planning capability.

Difficulty varies significantly: easy instances (01) achieve 100% success, while hard instances (11, 13, 15) achieve only 25-50%. The performance gap reflects challenges in state tracking, infrastructure management, and temporal ordering of multiple agents.

5.2.5 *Tetris Domain Results*

Tetris geometric reasoning achieves higher success (20-85%) than City Car. LLaMA3-8B leads at 85%, while Phi4-14B shows substantial improvement from City Car (25% to 65%), revealing specialized geometric reasoning strength. Convergence is faster in Tetris (1.4 avg iterations) than City Car (1.7), indicating geometric feedback provides clearer error signals.

Instance-15 achieves near-universal failure (0%) across all models and iterations, suggesting fundamental problem characteristics requiring novel approaches. Most other instances achieve 75%+ success, demonstrating strong general geometric reasoning capability.

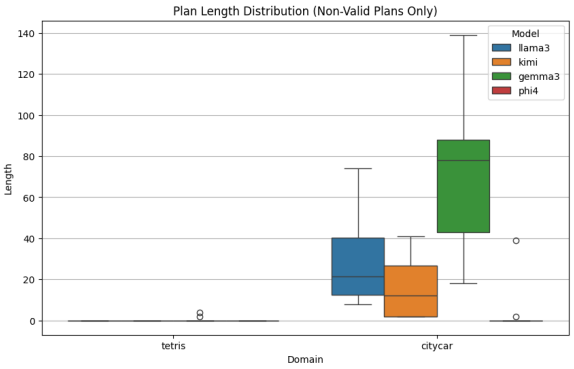

Figure 5: Not valid plan length

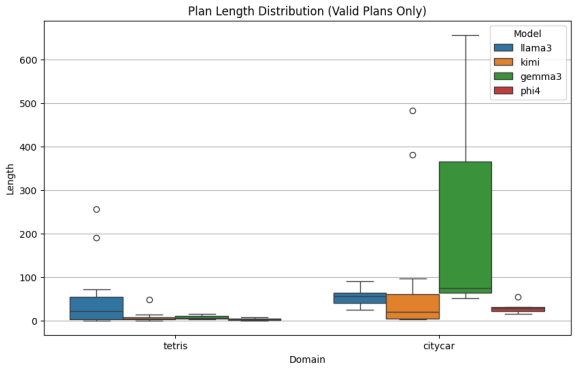

Figure 6: Valid plan length

Plan Length Analysis Figures 5 and 6 present comparative analysis of generated plan lengths across models and domains, distinguishing between invalid and valid plans. Invalid plans (Figure 5) reveal significant structural differences: Gemma3-27B generates substantially longer plans (median 40-80 actions for City Car) despite frequent validation failures, while valid plans (Figure 6) show more reasonable lengths (median 25-75 actions). This pattern suggests that while larger models may generate more verbose outputs, successful plans demonstrate relatively consistent length characteristics across architectures, with domain-specific variations reflecting inherent problem complexity rather than model limitations.

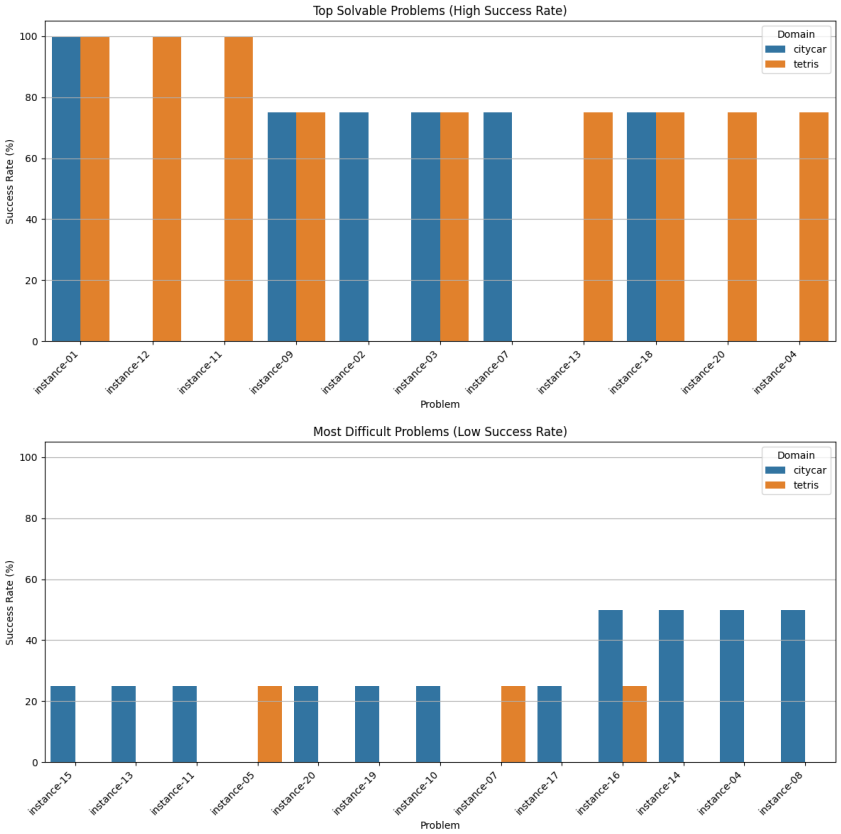

Figure 7: Most solvable problems (top) Most difficult problems (bottom)

5.2.6 Model Comparison and Rankings

Critical Methodological Caveat: All models were evaluated with prompts optimized iteratively on LLaMA3-8B, which was selected as baseline due to computational efficiency. This systematic bias advantages LLaMA3-8B and disadvantages models with different architectures. The performance ranking likely reflects prompt-architecture alignment rather than underlying capability differences.

LLaMA3-8B (82.5%) Achieves highest measured success, but this reflects extensive prompt tuning specifically on this model. Superior results should be interpreted as evidence of successful prompt optimization rather than planning superiority. When a model serves as development baseline with iterative refinement, performance advantages are expected.

Kimi-Dev-72B (80%) Achieves remarkably strong performance (80% both domains) with *zero* architecture-specific prompt customization. This consistency represents the most robust finding regarding generalization. With equivalent prompt engineering to LLaMA3-8B, Kimi-Dev-72B could plausibly match or exceed current LLaMA3-8B performance.

Phi4-14B (45%) Demonstrates striking domain specialization (65% Tetris vs. 25% City Car) despite receiving no prompt customization. Rapid convergence (1.27 avg iterations) indicates errors are largely syntactic rather than conceptual, suggesting targeted prompt engineering could substantially improve performance.

Gemma3-27B (17.5%) Shows consistently low performance across both domains. High iteration count (1.71) despite poor success suggests either fundamental limitations or severe prompt misalignment. Architecture-specific prompt engineering is required to draw conclusions about true capability.

6 Conclusions

6.1 Summary of Contributions

This work investigates the emergent planning and reasoning capabilities of modern Large Language Models through systematic evaluation across diverse architectures and planning domains. Our key contributions include:

1. **Comprehensive Empirical Framework**: A modular, reproducible framework for evaluating LLM planning capabilities that integrates advanced prompt engineering strategies with formal plan validation via the VAL validator.

2. **Extensive Model Evaluation**: Systematic comparative evaluation of four diverse open-source LLMs (LLaMA3-8B, Phi4-14B, Gemma3-27B, Kimi-Dev-72B) across two distinct planning domains with 20 problem instances each.

3. **Iterative Refinement Methodology**: Demonstration that iterative refinement with validation feedback significantly improves planning success rates, enabling LLMs to learn from failures within a single conversation session.

4. **Prompt Engineering Insights**: Validation of sophisticated prompt engineering techniques including chain-of-thought reasoning, few-shot learning, and iterative feedback incorporation.

5. **Distributed Computing Infrastructure**: Development of reproducible experimental pipelines suitable for large-scale execution on high-performance computing clusters.

6.2 Empirical Findings

Our comprehensive experiments on 640 planning task instances (4 models $\times$ 2 domains $\times$ 20 instances $\times$ 4 iteration configurations) reveal the following key empirical findings:

- **Iterative Refinement Effectiveness**: Iterative refinement with validation feedback provides substantial improvements in success rates across all tested models and domains. Most improvements occur within 1-2 refinement cycles, with strong diminishing returns beyond iteration 3. Models achieve average convergence between 1.27-1.71 iterations, far below the theoretical maximum of 4, indicating effective error-driven learning.

- **Model Capability Variation**: Planning capability is not simply a function of parameter count. LLaMA3-8B (8B parameters) outperforms Gemma3-27B (27B parameters) by 65 percentage points, demonstrating that architectural design, training methodology, and inductive biases substantially outweigh raw parameter scaling. Clear model stratification exists: LLaMA3 and Kimi (80%+) are production-ready, Phi4 (45%) is domain-limited, and Gemma3 (17.5%) is unsuitable for planning without additional approaches.

- **Domain-Specific Challenges**: Different planning domains expose distinct model strengths and weaknesses. City Car domain (multi-agent coordination) achieves lower success rates (50% mean) compared to Tetris (geometric reasoning: 62.5%

mean). Convergence is faster in Tetris (1.4 avg iterations) than City Car (1.7 iterations), suggesting geometric feedback provides clearer correction signals. Phi4's 40-point performance gap between domains (25% City Car vs. 65% Tetris) reveals specialized geometric reasoning capability with weak multi-agent coordination.

- **Problem Difficulty Distribution**: Success rate variation within domains is substantial. Easy instances (01, 11, 12 in Tetris; 01 in City Car) achieve near-universal success (75-100%), while hard instances (15 in both domains) achieve near-universal failure (0-5%) across all models and iterations. This suggests that problem structure difficulty transcends model selection, implying that certain problems require novel decomposition strategies or architectural innovations.

- **Robustness of In-Context Learning**: Models demonstrate effective in-context learning from validation feedback within single conversation sessions. Validation error messages successfully guide error correction, with high-confidence patterns (e.g., Phi4's rapid iteration convergence despite moderate overall success) suggesting that syntactic/minor semantic errors are readily correctable through feedback. Conversely, structural reasoning gaps (e.g., multi-agent coordination in poor performers) resist correction even with repeated feedback.

6.3 Future Work

This research opens several promising directions for future investigation:

1. **Expanded Model Coverage**: Evaluation of additional model architectures, including newer models and different parameter scales, to further understand scaling laws for planning capability.

2. **Domain Generalization**: Investigation of how planning skills transfer across different planning domains and problem types, including continuous domains and real-world scenarios.

3. **Hybrid Approaches**: Integration of LLM-based planning with classical planning solvers in LLM-modulo frameworks, combining the strengths of both paradigms.

4. **Fine-tuning Studies**: Investigation of whether domain-specific fine-tuning or instruction-tuning can further improve planning performance beyond prompt-based approaches.

5. **Theoretical Analysis**: Development of formal frameworks for understanding when and why LLMs succeed or fail at planning tasks, moving beyond purely empirical evaluation.

6. **Scalability to Complex Domains**: Extension to larger, more realistic planning problems with hundreds or thousands of actions and objects.

This work contributes to the growing body of research on LLM-based reasoning and provides a foundation for future investigations into the planning and reasoning capabilities of language models.

References

[1] McDermott, D. PDDL—The Planning Domain Definition Language.

[2] Stojnic, G., Koretic, D., & Pavlovic, V. Planning with Language Models Through a Generative World Model.

[3] Subbarao Kambhampati, Karthik Valmeekam, Lin Guan, Mudit Verma, Kaya Stechly, Siddhant Bhambri, Lucas Saldyt, Anil Murthy. Position: LLMs Can't Plan, But Can Help Planning in LLM-Modulo Frameworks.

[4] Hui Wei, Zihao Zhang, Shenghua He, Tian Xia, Shijia Pan, Fei Liu. PlanGenLLMs: A Modern Survey of LLM Planning Capabilities.

[5] Aske Plaat, Annie Wong, Suzan Verberne, Joost Broekens, Niki van Stein, Thomas Back. Reasoning with LLMs survey.

[6] Chan Hee Song, Jiaman Wu, Clayton Washington, Brian M. Sadler, Wei-Lun Chao, Yu Su. LLM-Planner: Few-Shot Grounded Planning for Embodied Agents with Large Language Models.

[7] Silin Meng, Yiwei Wang, Cheng-Fu Yang, Nanyun Peng, Kai-Wei Chang. LLM-A: Large Language Model Enhanced Incremental Heuristic Search on Path Planning

[8] Aidan Curtis, Nishanth Kumar, Jing Cao, Tomàs Lozano-Pèrez, Leslie Pack Kaelbling. Trust the PRoC3S: Solving Long-Horizon Robotics Problems with LLMs and Constraint Satisfaction.

A Model Architectures

A.1 LLaMA3-8B

LLaMA3 (Large Language Model Meta AI) is Meta's open-source large language model family. The 8-billion parameter variant provides a good balance between capability and resource efficiency, making it suitable for academic research environments. It demonstrates solid reasoning capabilities across diverse tasks while being computationally accessible.

A.2 Phi4-14B

Phi models are developed by Microsoft and are designed to demonstrate high-quality reasoning capabilities despite their relatively smaller size compared to other contemporary models. Phi4 represents the latest generation with 14 billion parameters, emphasizing efficiency and reasoning performance.

A.3 Gemma3-27B

Gemma is Google's family of lightweight open models, optimized for efficient inference. The 27-billion parameter variant offers increased capacity while maintaining reasonable computational requirements, balancing performance and efficiency.

A.4 Kimi-Dev-72B

Kimi is a large multilingual model with strong reasoning capabilities developed by Moonshot AI. The 72-billion parameter variant represents a larger-scale model suitable for complex planning tasks, with demonstrated strength in structured reasoning across multiple languages.

B Configuration File Examples

B.1 Main Configuration (config.yml)

```yaml
# Model and data paths configuration
MODEL_PATH: "src/models/Phi4"  # Llama3, Gemma3, Kimi or any
   other model directory
PROBLEMS_PATH: "src/data"        # Root directory containing
   domain folders (tetris, citycar, etc.)
MODEL_OUTPUT: "src/results"      # Alias for main.py
   compatibility

# Validation settings
VAL_PATH: "VAL/build/linux64/Release/bin"  # Path to VAL
   binaries directory
VAL_EXECUTABLE: "Validate"  # VAL executable name (note: capital
   V)
VAL_TIMEOUT: 300  # Seconds

# Model settings
MAX_TOKENS: 5000
TEMPERATURE: 0.6
TOP_K: 10

# Multi-GPU settings
DEVICE_MAP: "auto"        # Options: "auto", "balanced",
   "sequential", or specific mapping
MAX_GPU_MEMORY: null      # Per-GPU memory limit in GB (null for
   no limit)

# Execution settings
DEFAULT_ITERATIONS: 3  # For iterative validation
BATCH_SIZE: 1
```

C SLURM Submission Scripts

C.1 Example: LLaMA3 with 3 Iterations

```
#!/bin/bash
#SBATCH --account=IscrC_VisLLMs
#SBATCH --partition=boost_usr_prod
#SBATCH --time=24:00:00
#SBATCH --gres=gpu:1
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --cpus-per-task=8
#SBATCH --mem=64G
#SBATCH --job-name=llama3_iters_3
#SBATCH --output=llama3_iters_3.out
#SBATCH --error=llama3_iters_3.err

module load python/3.11.7

if [ -d "project_venv" ]; then
    VENV_DIR="project_venv"
elif [ -d "venv" ]; then
    VENV_DIR="venv"
else
    echo "ERROR: Virtual environment not found!"
    exit 1
fi

source $VENV_DIR/bin/activate

export PYTHONPATH="$(pwd)/src:$PYTHONPATH"
export LLM_PROJECT_ROOT="$(pwd)"

python src/main.py \
  --weights_path src/models/Llama3 \
  --problems_path src/data \
  --output_dir src/results \
  --batch \
  --max_iterations 3
```

D Prompt Templates

D.1 System Prompt

```
You are an expert automated PDDL planning assistant specializing
in complex domains like Tetris and Citycar, which involve
   explicit
coordinate systems, multi-parameter actions, negative
   preconditions,
and action costs.

Your primary objective is to find the **shortest and/or
   lowest-cost**
plan that achieves the goal.

When asked to return a plan, follow these rules unless explicitly
told otherwise:
- Output only the action sequence, one action per line, using the
  exact PDDL action syntax: (action-name param1 param2 ...).
- Do not include explanations, commentary, or extra metadata
   unless
  requested.
- Crucially: Verify that all action preconditions are respected
   in
  the current state and that the plan achieves all goal
     predicates
  while minimizing the total-cost.
```

D.2 Chain-of-Thought Template

```
Let's reason step-by-step, about the domain and problem
   descriptions
provided, to construct the optimal plan.

Output ONLY the corrected action sequence (one action per line).
```

D.3 Few-Shot Examples

D.3.1 Tetris Domain Examples

```
Example 1 - Move Single Square:
Instance:
(:objects squareA - one_square f0-0f f0-1f - position)
(:init
(clear f0-1f)
(at_square squareA f0-0f)
(connected f0-0f f0-1f)
(connected f0-1f f0-0f)
)
(:goal (at_square squareA f0-1f))

Plan:
(move_square f0-0f f0-1f squareA)


Example 2 - Move Two-Piece Block:
Instance:
(:objects straightB - two_straight f0-0f f1-0f f2-0f - position)
(:init
(clear f2-0f)
(at_two straightB f0-0f f1-0f)
(connected f1-0f f2-0f)
)
(:goal (at_two straightB f1-0f f2-0f))

Plan:
(move_two f0-0f f1-0f f2-0f straightB)
```

D.3.2 CityCar Domain Examples

```
Example 1 - Move Car via Existing Road:
Instance:
(:objects car0 - car junction0-0 junction1-0 - junction road0 -
   road)
(:init
(at_car_jun car0 junction0-0)
(clear junction1-0)
(road_connect road0 junction0-0 junction1-0)
(in_place road0)
)
(:goal (at_car_jun car0 junction1-0))

Plan:
(move_car_in_road junction0-0 junction1-0 car0 road0)
(move_car_out_road junction0-0 junction1-0 car0 road0)
```

```
Example 2 - Car Start with Road Construction:
Instance:
(:objects car1 - car junction0-0 junction0-1 - junction garage0
   - garage road1 - road)
(:init
(starting car1 garage0)
(at_garage garage0 junction0-0)
(clear junction0-0)
(clear junction0-1)
(same_line junction0-0 junction0-1)
(not (in_place road1))
)
(:goal (at_car_jun car1 junction0-1))

Plan:
(car_start junction0-0 car1 garage0)
(build_straight_oneway junction0-0 junction0-1 road1)
(move_car_in_road junction0-0 junction0-1 car1 road1)
(move_car_out_road junction0-0 junction0-1 car1 road1)


Example 3 - Car Arrival and Road Destruction:
Instance:
(:objects car0 - car junction1-0 junction2-0 - junction road2 -
   road)
(:init
(at_car_jun car0 junction2-0)
(road_connect road2 junction1-0 junction2-0)
(in_place road2)
)
(:goal
(and
(arrived car0 junction2-0)
(not (in_place road2))
)
)

Plan:
(car_arrived junction2-0 car0)
(destroy_road junction1-0 junction2-0 road2)
```